Deep Learning

Assignment #02

Name: Amama Sajjad
Class: 7E
Roll no: 23F-6014

Question #01

(A)

1. 11x11 convolution, stride 4 ⟶ AlexNet

2. Repeated 3x3 convolution: VGGNet.

3. Increasing feature maps while reducing spatial resolution: General CNN standard.

4. bypassing layer: ResNet

5. Multiple convolution branches combined: Inception

6. Replacing FC layers with GAP (global average pooling): NIN (Network in Network).

(B)

1. AlexNet:

Aggressively downsamples high-resolution inputs early in the network to reduce memory usage and computational load. It became feasible to train on early GPUs.

2. VGGNet:

Large kernels (5x5, 7x7) are compututionally expensive and increase learnable parameters. Stacking 3x3 filters achieves the same receptive field with far fewer parameters and deeper non-linear combinations.

CamScanner

3. General CNN:

As spatial information is compressed, the network needs diverse feature type to represent the image. The nework needs higher number of channels to encode complex semantic features without degrading computational cost.

4. ResNet:

Limitation addressed, the vanishing gradient problem and the degradation problem where adding more layers to a plain network causes training accuracy to saturate and then rapidly degrade.

5. Inception:

Chosing the optimal kernel size is difficult because visual features appear at different positions. Inception solves it by processing features at multiple scales natively in parallel.

6. Network in Network:

Fully connected layers contain the (almost >80%) of the network's parameter → leading to overfitting & memory intensive. NIN solved it using Global Average Pooling.

CamScanner

Part (C)

Architectural Comparison

Feature	AlexNet	Inception
Network-depth	8 layers	22 layers
Filter-design	Initially large (11×11, 5×5) later small (3×3).	multiscale (1×1, 3×3, 5×5)
Feature-extraction	sequential, coarse to fine	Parallel multi-scale extraction.
Information flow	simple forward pass	forward pass + auxilary classifier.
computationally efficiency	Low (Heavy FC)	high (1×1 dimension reduction)
Main Innovation	Proved deep learning on GPUs	Multi-branch ↳ inception module.

Part (D)

Plain vs Shortcut Network (same depth):

Even with identical layer count, both networks work differently. The main reason for this is learnable parameters depend on gradiant path.

A very deep plain network suffers from the degradation

CamScanner

problem During backpropagation repeated multiplication of small weights causes gradients to shrink to near zero → vanishing gradients stopping learning in early stages.

In a network with short cut connections, gradients flow freely backword through the identity mappings

$$(H(x) = F(x) + x)$$

bypassing the stacked convolutional layers entirely. This allows the shortcut network to learn easily, preventing accuracy degradation.

Part (E)

Stacking 3x3 (three) convolutions yield the same effective receptive field as a single 7x7 convolution but offering two major architectural benefits:

(i) Fewer Parameters:

Three 3x3 layer with C channels require $3 \times (3^2 \times C^2) = 27C^2$ weights.

A single 7x7 layer with C channels requires $7^2 \times C^2 = 49C^2$ weights.

(ii) Greater non-linearity:

Three layers incoporate three seperate activation functions (ReLu) rather than one. This helps the decision boundary more good at discremination.

Why not inception:

Inception works with capturing visual information at multiple spatial scales simultaneously. While 3x3 (3 layers) work sequentially. An inception module applies 1x1, 3x3 and 5x5 convolution in parallel. This allows the network to adapt to local patterns and broader spatial context in the exact same feature map (level).

CamScanner

(F)

1. AlexNet (2012):

↳ Addressed the lack of computational scalability. It did so by parallel training across GPU. It also solved early vanishing gradiant by introducing the ReLu activation function.

2. Inception (2014):

↳ Addressed the extreme computational cost and increasing parameter problem. It did so by removing fully connected layers and using 1x1 convolutions to reduce dimensionality. before expensive filtering operations.

Question#02

(A)

Conv Parameters P_c : $(K \times K \times C_{in} \times C_{out}) + C_{out}$

→ conv 1 :

$(11 \times 11 \times 3 \times 96) + 96 = 34848 =$ **34944**

→ ~~pool 1~~:

→ conv 2:

$(5 \times 5 \times 96 \times 256) + 256 = 614400 + 256 =$ **614656**

→ conv 3:

$(3 \times 3 \times 256 \times 384) + 384 = 884,736 + 384 =$ **885,120**

→ conv 4:

$(3 \times 3 \times 384 \times 384) + 384 = 1,327,104 + 384 =$ **1,327,488**

→ conv 5:

$(3 \times 3 \times 384 \times 256) + 256 = 884,736 + 256 =$ **884,992**

CamScanner

(B)

Output size

- **Conv 1:**

$$\text{output: } \left[\frac{227-11+0}{4}\right]+1 = 55 \qquad \text{output size} = 55 \times 55 \times 96$$

- **Pool 1:**

$$\left[\frac{55-3}{2}\right]+1 = 27 \qquad \text{output size} = 27 \times 27 \times 96$$

- **Conv 2:**

$$\left[\frac{27-5+4}{1}\right]+1 = 27 \qquad \text{output size} = 27 \times 27 \times 256$$

- **pool 2:**

$$\left[\frac{27-3}{2}\right]+1 = 13 \qquad \text{output size} = 13 \times 13 \times 256$$

- **Conv 3:**

$$\left[\frac{13-3+2}{1}\right]+1 = 13 \qquad \text{output size} = 13 \times 13 \times 384$$

- **Conv 4:**

$$\left[\frac{13-3+2}{1}\right]+1 = 13 \qquad \text{output size} = 13 \times 13 \times 384$$

- **Conv 5:**

$$\left[\frac{13-3+2}{1}\right]+1 = 13 \qquad \text{output size} = 13 \times 13 \times 256$$

- **pool 3:**

$$\left[\frac{13-3}{2}\right]+1 = 6 \qquad \text{output volume} = 6 \times 6 \times 256$$

(C)

FC parameters

1. **FC 1:** $(9216 * 4096) + 4096 = 37,748,736 + 4096 = 37752832$
2. **FC2:** $(4096 * 4096) + 4096 = 16777216 + 4096 = 16781312$
3. **FC3:** $(4096 \times 1000) + 1000 = 4096000 + 1000 = 4097000$

CamScanner

Part D

Total Trainable parameters

34,944 + 614,656 + 885,120 + 1,327,488 + 884,992 + 37,752,832 + 16,781,312 + 4097000

⟶ 62, 378, 344 parameters.

Question #03

Layer by Layer:

- **Conv 1:**

  input image: 32x32x1

  processed : 5x5 filter

  output: 28x28x6 ⟶ 6 filters.

  weights: 5x5x1 x 6 filters = 150

  biases: 6

  Trainable parameters : 156

- **Avg Pool 1:**

  pooling does not have trainable parameters it just down samples.

- **Conv 2:**

  input: 14 x 14 x 6

  filter : 5 x 5

  output: 10 x 10 x 16

  weights: 5 x 5 x 6 (input channel) x 16 (filters) = 2400

  biases: 16

Trainable Parameters: 2,400 + 16 = 2416

CamScanner

- **Avg Pool 2:**

  Trainable parameters = 0

- **FC 1:**

  input : 5 x 5 x 16 = 400 input features
  output : 120 neurons.
  weights : 400 x 120 = 4800 + 120(biases) = 48,120 (Trainable parameters.)

- **FC2:**

  input : 120 neurons. output : 84 neurons.
  weights : 120 x 84 = 10,080
  biases : 84
  Trainable parameters : 10,080 + 84 = 10,164

- **Softmax:**

  input = 84 output = 10 neurons.
  weights : 10 x 84 = 840
  biases : 10
  Trainable Parameters : 840 + 10 = 850

Total:

= 156 + 2416 + 48120 + 10164 + 850

= 61706 answer.

CamScanner